Unsupervised Deep Learning

Week 2: Autoregressive Models

October 2026

Magda Gregorová - magda.gregorova@thws.de



Licensed according to CC-BY-SA 4.0

Lecture Overview

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

1. Dependence and Structure
2. Autoregressive Models
3. Training and Sampling
4. Masked Architectures

Where We Left Off

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Model	Problem for images
Histogram	2^{784} cells, most empty
Independent pixels	speckle - no dependence
One Gaussian	one blob - too rigid

Need: dependence between variables - without a giant table
Today: how to model dependence - and one neural network to do it

Probability Rules

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Joint: $p(x, y)$

Marginal: $p(x) = \sum_y p(x, y)$

Conditional: $p(y \mid x) = p(x, y)/p(x)$

Product rule: $p(x, y) = p(x)\, p(y \mid x)$

Independence: $p(y \mid x) = p(y)$
$\Leftrightarrow \; p(x, y) = p(x)\, p(y)$

	water	beer	wine	marginal
soup	0.05	0.00	0.00	0.05
pizza	0.05	0.25	0.05	0.35
pasta	0.05	0.00	0.20	0.25
burger	0.05	0.15	0.00	0.20
salad	0.10	0.00	0.05	0.15
marginal	0.30	0.40	0.30	1.00

Two Extremes

Model	$p(\mathbf{x})$	Assumption
Independent	$\prod_{i=1}^{D} p(x_i)$	strong - no dependence
Chain rule	$\prod_{i=1}^{D} p(x_i \mid \mathbf{x}_{<i})$	none - exact

Chain rule: product rule repeated, shorthand $\mathbf{x}_{<i} = (x_1, \ldots, x_{i-1})$

$$p(\mathbf{x}) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2) \cdots p(x_D \mid x_1, \ldots, x_{D-1})$$

Any order is exact: $p(\text{dish})\, p(\text{drink} \mid \text{dish}) = p(\text{drink})\, p(\text{dish} \mid \text{drink})$

D variables $\Rightarrow D!$ orders with same $p(\mathbf{x})$

Conditional Independence

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

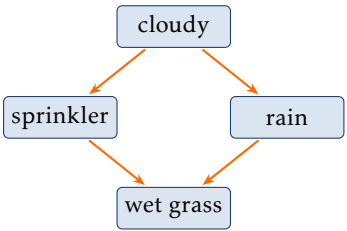
Definition: $x \perp\!\!\!\perp y \mid z \iff p(x \mid y, z) = p(x \mid z) \iff p(x, y \mid z) = p(x \mid z)\, p(y \mid z)$

Example: weather

- x tomorrow, y yesterday, z today
- $p(\text{tomorrow} \mid \text{today}, \text{yesterday}) = p(\text{tomorrow} \mid \text{today})$
- Given today, yesterday adds nothing

Not the same as independence: yesterday and tomorrow are dependent when today is unknown

Graphical Models



- **Node:** variable
- **Arrow:** factor conditioned on this variable
- **Cloudy:** causes rain and sprinkler use
- **Wet grass:** rain and sprinkler only
- **Independence:** $S \perp\!\!\!\perp R \mid C$

General form: $p(\mathbf{x}) = \prod_{i=1}^{D} p(x_i \mid \text{parents}(x_i))$

Here: $p(C, S, R, W) = p(C)\, p(S \mid C)\, p(R \mid C)\, p(W \mid S, R)$

Markov Chain

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

$$p(\mathbf{x}) = p(x_1) \prod_{i=2}^{D} p(x_i \mid x_{i-1})$$

- **Graph:** chain - one parent each
- **Assumption:** $x_i \perp\!\!\!\perp \mathbf{x}_{<i-1} \mid x_{i-1}$ - last variable only
- **Order k:** condition on last k variables

Weather: today decides tomorrow
Text: n-gram model - next word from last k words

Counting Parameters

thws Technical University of Applied Sciences Würzburg-Schweinfurt

Binary variable with m parents: 2^m parameters (one prob per parent config)

Model: sum over all factors

Model	**Parents of x_i**	**Parameters**	$D = 784$
Independent	0	$1 + 1 + \cdots + 1 = D$	784
Markov, order 1	1	$1 + 2 + \cdots + 2 = 2D - 1$	1 567
Sprinkler	0, 1, 1, 2	$1 + 2 + 2 + 4 = 9$	—
Chain rule	$i - 1$	$1 + 2 + 4 + \cdots + 2^{D-1} = 2^D - 1$	$\approx 10^{236}$

Text: V words, context = last k words $\Rightarrow V^k$ configurations, each with $V - 1$ probs

$V = 50\,000$, $k = 3 \Rightarrow$ table with $V^k(V-1) \approx 6 \cdot 10^{18}$ entries

Where Does Order Come From?

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Data	Order	Origin
Time series	time	natural - causal
Text	reading order	natural - words or tokens
Images	raster scan	convention - arbitrary

- **Long history matters:** text - far back, images - pixel above is 28 steps back
- **No domain knowledge** for sparse structure (unlike sprinkler)
- **Markov assumption** throws information away

Need: full past $\mathbf{x}_{<i}$ as condition

Probability as a Function

Variable $y \in \mathcal{Y}$, **condition** $x \in \mathcal{X}$

- **Probability:** function of y $\quad p : \mathcal{Y} \to [0,1]$ or $\mathbb{R}_{\geq 0}$
- **Conditional:** one function of y per x

	List	**Formula**
$p(y)$	$y \mapsto p(y)$	$p(y \mid \boldsymbol{\phi})$
$p(y \mid x)$	$(x, y) \mapsto p(y \mid x)$	$p(y \mid \boldsymbol{\phi}_x)$

List: discrete (countable) vars
Categorical: $p(y = k \mid \boldsymbol{\phi}) = \phi_k$, $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_V)$

Formula: continuous vars cannot list
Gaussian: $p(y \mid \boldsymbol{\phi}) = \mathcal{N}(y \mid \mu, \sigma^2)$, $\boldsymbol{\phi} = (\mu, \sigma)$

Chain Rule Factors: One Function per Condition

thws Technical University of Applied Sciences Würzburg-Schweinfurt

Factor: $p(x_i \mid \mathbf{x}_{<i})$ - variable $y = x_i$, condition $x = \mathbf{x}_{<i}$

Family fixed by type of x_i: Bernoulli, categorical, Gaussian, mixture, ...
functions in the family determined by parameters ϕ

But different ϕ_x for every condition $x = \mathbf{x}_{<i}$

- Many: 2^{i-1} conditions for binary var; infinitely many for continuous $\mathbf{x}_{<i}$
- Each ϕ_x needs to be inferred - histogram or Gaussian fit per condition
- Some conditions never observed - nothing to fit

Need: compact rule $\mathbf{x}_{<i} \mapsto \phi_x$ for any past

Amortisation: Learn the Rule

Instead of inferring ϕ_x separately for every x **learn** f_θ (NN) to predict:

$$\text{predict } f_\theta(x) = \phi_x \rightarrow \text{use it to get prob } p(y \mid \phi_x)$$

- **One function:** f_θ computes ϕ_x for every x
- **Shared weights θ:** learned once from all examples for all conditions
- **Inductive bias:** similar $x \rightarrow$ similar ϕ_x (smooth f_θ)
- **Unseen or continuous x:** f_θ still gives ϕ_x

Autoregressive: $y = x_i$, $x = \mathbf{x}_{<i}$

$$p_\theta(\mathbf{x}) = \prod_{i=1}^{D} p(x_i \mid \phi_i), \qquad \phi_i = f_\theta(\mathbf{x}_{<i})$$

Maximum Likelihood

thws Technical University of Applied Sciences Würzburg-Schweinfurt

Data: $\mathcal{D} = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N)}\}$ **Model:** $p_{\boldsymbol{\theta}}(\mathbf{x}) = \prod_{i=1}^{D} p(x_i \mid \boldsymbol{\phi}_i)$, $\boldsymbol{\phi}_i = f_{\boldsymbol{\theta}}(\mathbf{x}_{<i})$

Learn: $\boldsymbol{\theta}$ that make the data likely

$$\mathcal{L}(\boldsymbol{\theta}) = \text{NLL} = -\frac{1}{N}\sum_{n=1}^{N} \log p_{\boldsymbol{\theta}}(\mathbf{x}^{(n)}) = -\frac{1}{N}\sum_{n=1}^{N}\sum_{i=1}^{D} \log p(x_i^{(n)} \mid \boldsymbol{\phi}_i^{(n)})$$

- **Product** of factors $\Rightarrow$ **sum** of log-factors
- **Minimise** by gradient descent on $\boldsymbol{\theta}$
- **Week 1:** counting, sample mean and variance - NLL for categorical / Gaussian

Maximum Likelihood and KL Divergence

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

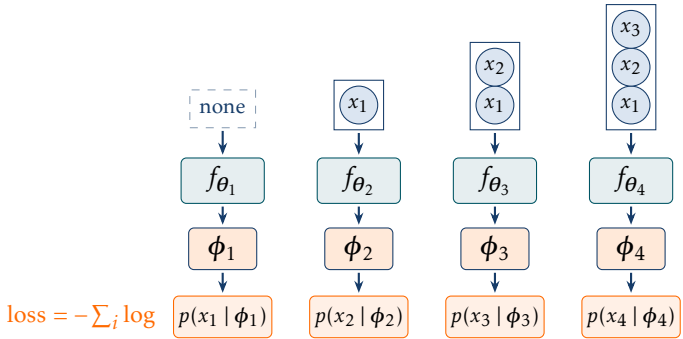
$$\mathrm{KL}(p_{\text{data}} \| p_\theta) = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\left[\log \frac{p_{\text{data}}(\mathbf{x})}{p_\theta(\mathbf{x})}\right] = \underbrace{\mathbb{E}_{p_{\text{data}}}[\log p_{\text{data}}(\mathbf{x})]}_{\text{constant in } \theta} - \mathbb{E}_{p_{\text{data}}}[\log p_\theta(\mathbf{x})]$$

- **KL** ≥ 0, zero iff $p_\theta = p_{\text{data}}$ - distance-like, not symmetric
- **Minimise KL** $\Leftrightarrow$ minimise $-\mathbb{E}_{p_{\text{data}}}[\log p_\theta(\mathbf{x})]$
- **NLL on data:** sample average estimate of this expectation

Maximum likelihood = fit p_θ close to p_{data} in KL

Bits per dimension: NLL $/(D \ln 2)$

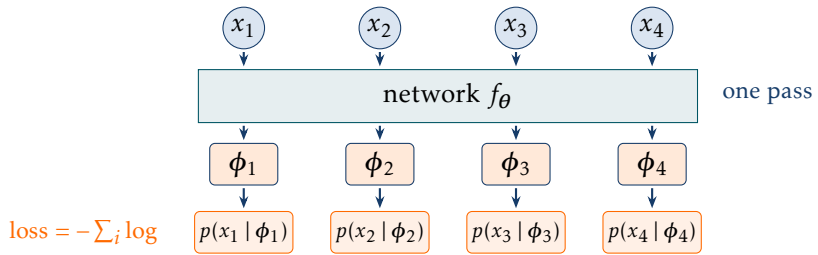
Training: Teacher Forcing



- **Network** f_{θ_i} for every i: input $\mathbf{x}_{<i}$, output ϕ_i
- **Loss:** sum of $-\log p(x_i \mid \phi_i)$ over i
- **Teacher forcing:** past $\mathbf{x}_{<i}$ from data, not from the model

Cost: D networks, D passes, no shared weights → Can we do better?

Training: All Conditionals in Parallel



- **Pros:** one pass for all ϕ_i - shared weights θ
- Challenge: network sees all of $\mathbf{x}$ - ϕ_i sees x_i

Need: ϕ_i depends on $\mathbf{x}_{<i}$ only - inputs must be cut

Masking: One Network, Right Dependencies

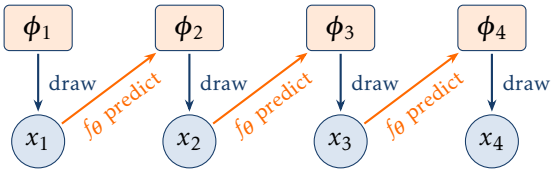
- **Target:** ϕ_i depends on x_j only for $j < i$
- **Mask:** fixed 0/1 matrix, $\mathbf{W} \odot \mathbf{M}$
- **Cuts** every path from x_j, $j \geq i$, to ϕ_i
- **Single network:** shared weights, one pass
- **Order** of variables = structure of masks

output \ input	x_1	x_2	x_3	x_4
ϕ_1	0	0	0	0
ϕ_2	1	0	0	0
ϕ_3	1	1	0	0
ϕ_4	1	1	1	0

Single network, shared weights, one pass - and ϕ_i sees $\mathbf{x}_{<i}$ only

Three prototypes: MADE (weights) - PixelCNN (kernels) - causal Transformer (attention)

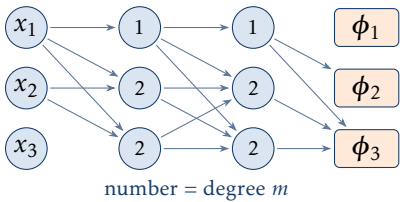
Sampling: One Variable at a Time



- **Ancestral sampling:** $x_1 \sim p(x_1 \mid \phi_1)$, then $\phi_2 = f_\theta(x_1)$, $x_2 \sim p(x_2 \mid \phi_2)$, ...
- D **network calls** - sequential, no parallelism

Likelihood: fast - one pass Sampling: slow - D passes

MADE: Masked Fully Connected Network



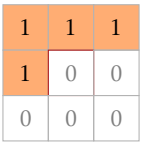
- **MLP** with masked weights $\mathbf{W} \odot \mathbf{M}$
- **Degree** $m \in \{1, \ldots, D-1\}$ every hidden unit
- **Input** $x_j \rightarrow$ **hidden:** allowed if $m \geq j$
- **Hidden** $\rightarrow$ **hidden:** allowed if $m_{\text{next}} \geq m_{\text{prev}}$
- **Hidden** $\rightarrow \phi_i$**:** allowed if $i > m$

Output ϕ_i sees only $x_{<i}$ - guaranteed by the mask

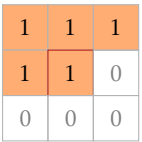
Germain et al.: MADE: Masked Autoencoder for Distribution Estimation. ICML 2015

PixelCNN: Masked Convolution

thws Technical University of Applied Sciences Würzburg-Schweinfurt



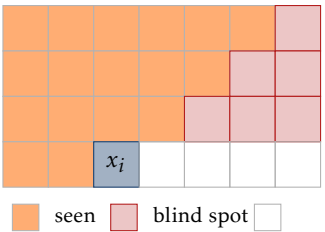
type A



type B

- **Convolutions:** inductive bias, shared parameters
- **Raster scan:** kernel sees pixels above and to the left
- **Type A:** first layer - centre pixel excluded
- **Type B:** later layers - centre included

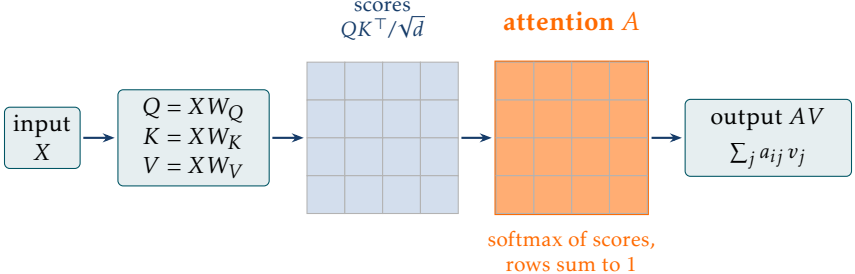
Blind spot: pixels to the upper right are missed



- **Stacked masked convolutions:** receptive field grows
- **Row k above:** reaches only k pixels to the right
- Blind spot: upper-right pixels never seen
- **Conditional less expressive:** past info lost
- **Fix:** horizongal & vertical stack - gated PixelCNN

van den Oord et al.: Pixel Recurrent Neural Networks. ICML 2016
van den Oord et al.: Conditional Image Generation with PixelCNN Decoders. NeurIPS 2016

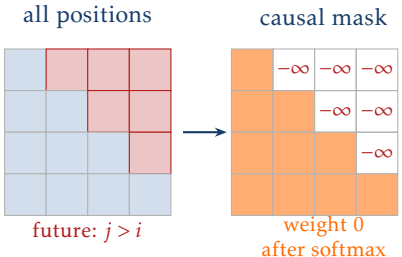
Attention



- **Query, key, value** per position - linear maps of the input
- **Score** $q_i \cdot k_j$ - how much position i looks at j
- **Softmax** per row - weights a_{ij}, sum 1
- **Output** i**:** weighted sum of values v_j over all j

Vaswani et al.: Attention Is All You Need. NeurIPS 2017

Causal Transformer



- Problem: output i sees x_j, $j \geq i$ - copy
- **Causal mask:** scores $j > i$ set to $-\infty$
- **Weights** exactly 0 after softmax
- **Shift input** by one - ϕ_i from $\mathbf{x}_{<i}$
- **Output:** softmax over tokens

Tokens: words, characters, pixels - **LLMs:** causal Transformers on text

Autoregressive Models: Pros and Cons

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

Pros

- Exact likelihood
- Stable training - plain NLL
- Any data type, discrete and conti
- Scales well - Transformers

Cons

- Slow sampling - D sequential steps
- Order must be chosen
- Errors accumulate in sampling
- No compact code of the data

Summary

- **Dependence:** independent - Markov - graphical - full chain rule
- **Chain rule:** exact, any order - factors depend on the whole past
- **Problem:** one function per condition - too many, none for unseen
- **Amortisation:** one network $\phi_i = f_\theta(\mathbf{x}_{<i})$ with shared weights
- **Training:** maximum likelihood = KL from data to model, one parallel pass
- **Sampling:** one variable at a time - slow
- **Masking:** MADE, PixelCNN, causal Transformer